

第4章 循环结构

4.1 while 循环

4.1.1 while 语句

- 格式：while (<expression>) {<codeblock>}
 - <codeblock> 称为**循环体**
- 原理：
 - i. 计算 <expression> 的值
 - 若为 false ，则退出循环体
 - 若为 true ，则执行 <codeblock> 中的语句，之后回到第1步

4.1.2 do-while 语句

- 作用：确保循环体至少被执行了1次
- 格式：do {<codeblock>} while (<expression>)
- 原理：先执行1次循环体，再按照普通 while 语句的方式执行

4.2 for 循环

4.2.1 for 语句

- 格式：for (<expression 1>; <expression 2>; <expression 3>) {<codeblock>}
 - for () 称为**循环控制行**
- 原理：
 - i. 执行 <expression 1>
 - ii. 计算 <expression 2> 的值
 - 若为 false ，则退出循环体
 - 若为 true ，则执行 <codeblock> 中的语句，之后跳到第3步
 - iii. 执行 <expression 3> ，之后回到第2步
- 作用：便于设置循环次数
 - <expression 1> 用于初始化循环变量
 - <expression 2> 用于设置循环条件
 - <expression 3> 用于对循环变量进行操作
 - 常用 i ， j ， k 作为循环变量名，例如按定义实现阶乘：

```
// Suppose that n is known
int result = 1;
for (int i = 1; i <= n; i++)
    result *= i;
```

- 循环变量与销毁：定义在循环控制行内的变量将在循环结束后销毁，不再可用
 - 例如上面阶乘中，for 循环退出之后变量 i 被销毁，之后使用 i 会报错
 - 定义在循环控制行外的变量不受影响，循环结束后仍可使用，并且保留循环过程中的变化

4.2.2 ，运算符

- 格式：<expression 1>, <expression 2>, ..., <expression n>;
- 执行：从左到右依次计算各个表达式的值
 - 整个语句的返回值为 <expression n> 的值
 - , 运算符是所有运算符中优先级最低的
- 作用：整合多个表达式为单个语句

4.2.3 杂项

- 表达式省略：for 循环控制行中3个表达式都可以省略
 - 默认空表达式结果为 true
 - ; 不可以省略
- 死循环：若 <expression 2> 的结果一直为 true 则循环体不能退出
 - 最简单的死循环为 for (; ;); ，即循环控制行和循环体都为空
 - 可通过下述中途退出方式避免死循环
- 循环嵌套：允许 for 循环和 while 循环互相嵌套
 - 在外层循环的每个循环周期，相邻内层循环执行所有循环周期
 - 注意循环变量在内外层循环间的作用，如果不希望互相干扰，要设置不同的循环变量

4.3 循环的中途退出

- 退出全部循环周期：循环体中读到 break; 语句会结束循环
- 退出单个循环周期：循环体中读到 continue; 语句会直接忽略本周期的语句，直接进入计算 <expression 2> 并判断的环节
- 在循环嵌套中只会退出单层循环的周期

4.4 相关算法

4.4.1 枚举法/遍历法

- 定义：列举所有可能是解的方案，并进行检验和筛选
- 优点：简单易理解；便于保证正确性
- 缺点：处理问题的时间效率差
- 优化：减少解的选择

4.4.2 贪婪法

- 定义：寻求局部最优，之后组合局部最优成为全局最优
- 优点：能快速缩小问题的规模
- 缺点：使用情况有限
 - 局部最优的组合可能不符合问题条件
 - 局部最优的组合可能不是全局最优